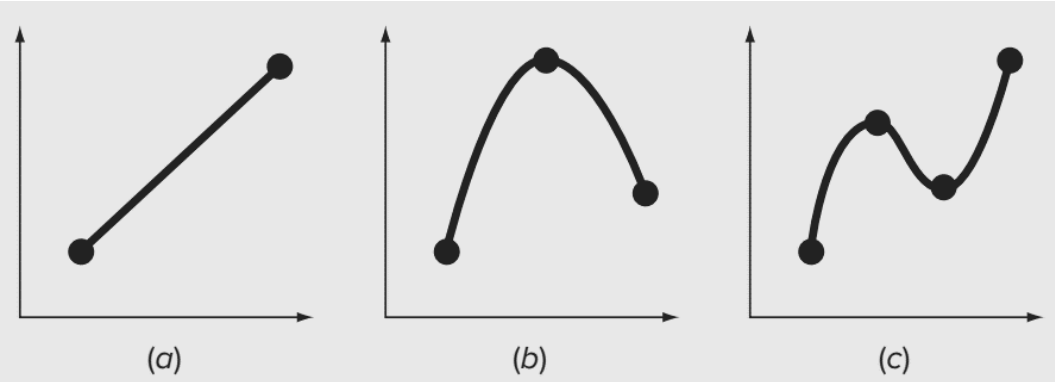
LAB 7: Polynomial Interpolation

Objective: To obtain the linear/polynomial interpolation using Newton/Lagrange's polynomial.

Theory: You will frequently have occasion to estimate intermediate values between precise data points. The most common method used for this purpose is polynomial interpolation. The general formula for an (n − 1)th-order polynomial can be written as-

$$f(x) = a_1 + a_2 x + a_3 x^2 + \cdots + a_n x^{n-1}$$

For n data points, there is one and only one polynomial of order (n − 1) that passes through all the points. For example, there is only one straight line (i.e., a first-order polynomial) that connects two points. Similarly, only one parabola connects a set of three points. Polynomial interpolation consists of determining the unique (n−1) order polynomial that fits n data points. This polynomial then provides a formula to compute intermediate values.



Examples of interpolating polynomials: (*a*) first-order (linear) connecting two points, (*b*) second-order (quadratic or parabolic) connecting three points, and (*c*) third-order (cubic) connecting four points.

A straightforward way for computing the coefficients, is based on the fact that n data points are required to determine the n coefficients.

Problem Statement. Suppose that we want to determine the coefficients of the parabola, $f(x) = p_1 x^2 + p_2 x + p_3$, that passes through the last three density values from Table 17.1:

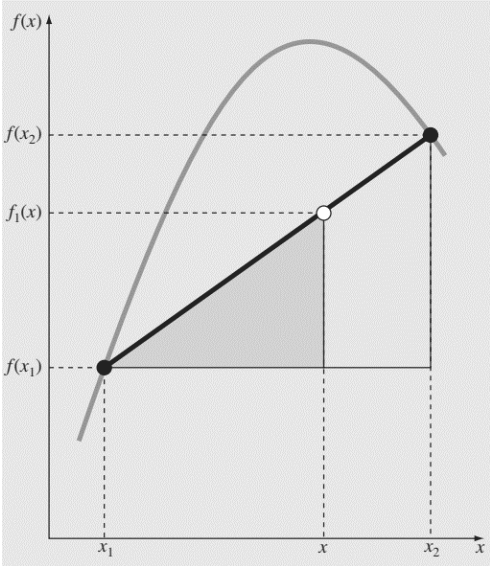
$$x_1 = 300 \quad f(x_1) = 0.616$$
$$x_2 = 400 \quad f(x_2) = 0.525$$
$$x_3 = 500 \quad f(x_3) = 0.457$$

This results in,

$$\begin{bmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ x_3^2 & x_3 & 1 \end{bmatrix} \begin{Bmatrix} p_1 \\ p_2 \\ p_3 \end{Bmatrix} = \begin{Bmatrix} f(x_1) \\ f(x_2) \\ f(x_3) \end{Bmatrix}$$

Such matrices are very ill-conditioned. That is, their solutions are very sensitive to round off errors. There are methods called, Newton's interpolating polynomial and Lagrange's interpolating polynomial, we can get rid of above problem.

The simplest form of interpolation is to connect two data points with a straight line. This technique, called linear interpolation, is depicted graphically in Fig.



Using similar triangles,

$$\frac{f_1(x) - f(x_1)}{x - x_1} = \frac{f(x_2) - f(x_1)}{x_2 - x_1}$$

which can be rearranged to yield

$$f_1(x) = f(x_1) + \frac{f(x_2) - f(x_1)}{x_2 - x_1}(x - x_1)$$

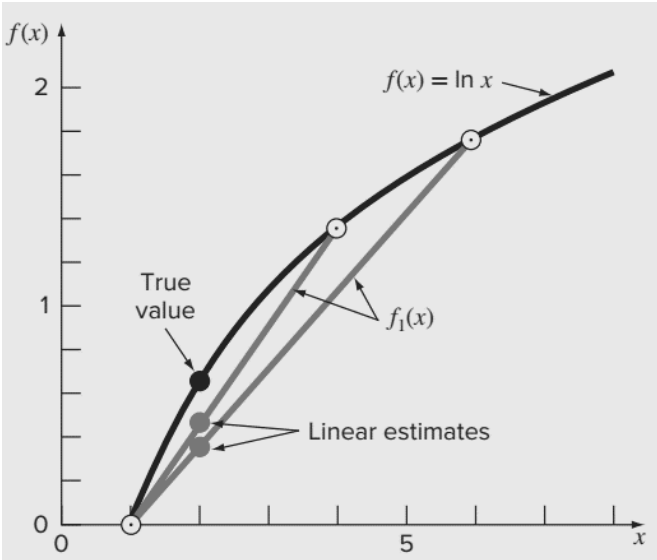
Problem Statement. Estimate the natural logarithm of 2 using linear interpolation. First, perform the computation by interpolating between ln 1 = 0 and ln 6 = 1.791759. Then, repeat the procedure, but use a smaller interval from ln 1 to ln 4 (1.386294). Note that the true value of ln 2 is 0.6931472.

Solution. We use Eq. (17.5) from $x_1 = 1$ to $x_2 = 6$ to give

$$f_1(2) = 0 + \frac{1.791759 - 0}{6 - 1}(2 - 1) = 0.3583519$$

which represents an error of $\varepsilon_t = 48.3\%$. Using the smaller interval from $x_1 = 1$ to $x_2 = 4$ yields

$$f_1(2) = 0 + \frac{1.386294 - 0}{4 - 1}(2 - 1) = 0.4620981$$



17.2.2 Quadratic Interpolation

The error in Example 17.2 resulted from approximating a curve with a straight line. Consequently, a strategy for improving the estimate is to introduce some curvature into the line connecting the points. If three data points are available, this can be accomplished with a second-order polynomial (also called a quadratic polynomial or a parabola). A particularly convenient form for this purpose is

$$f_2(x) = b_1 + b_2(x - x_1) + b_3(x - x_1)(x - x_2) \tag{17.6}$$

A simple procedure can be used to determine the values of the coefficients. For b_1, Eq. (17.6) with $x = x_1$ can be used to compute

$$b_1 = f(x_1) \tag{17.7}$$

Equation (17.7) can be substituted into Eq. (17.6), which can be evaluated at $x = x_2$ for

$$b_2 = \frac{f(x_2) - f(x_1)}{x_2 - x_1} \tag{17.8}$$

Finally, Eqs. (17.7) and (17.8) can be substituted into Eq. (17.6), which can be evaluated at $x = x_3$ and solved (after some algebraic manipulations) for

$$b_3 = \frac{\dfrac{f(x_3) - f(x_2)}{x_3 - x_2} - \dfrac{f(x_2) - f(x_1)}{x_2 - x_1}}{x_3 - x_1} \tag{17.9}$$

Problem Statement. Employ a second-order Newton polynomial to estimate ln 2 with the same three points used in Example 17.2:

$$x_1 = 1 \qquad f(x_1) = 0$$

$$x_2 = 4 \qquad f(x_2) = 1.386294$$

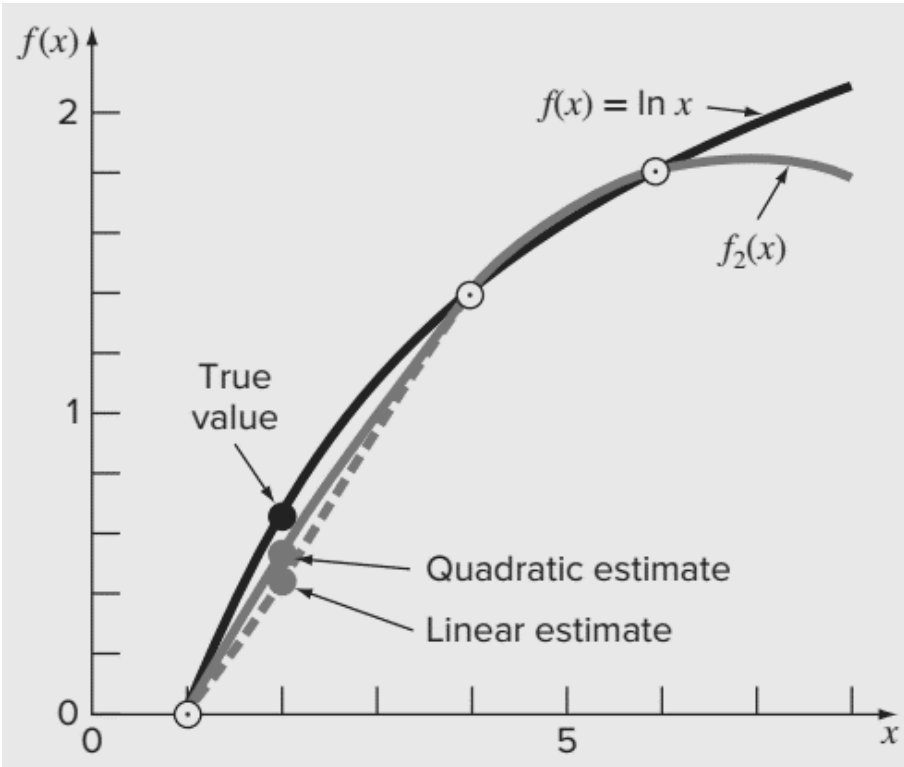
$$x_3 = 6 \qquad f(x_3) = 1.791759$$

Solution. Applying Eq. (17.7) yields

$$b_1 = 0$$

Equation (17.8) gives

$$b_2 = \frac{1.386294 - 0}{4 - 1} = 0.4620981$$



and Eq. (17.9) yields

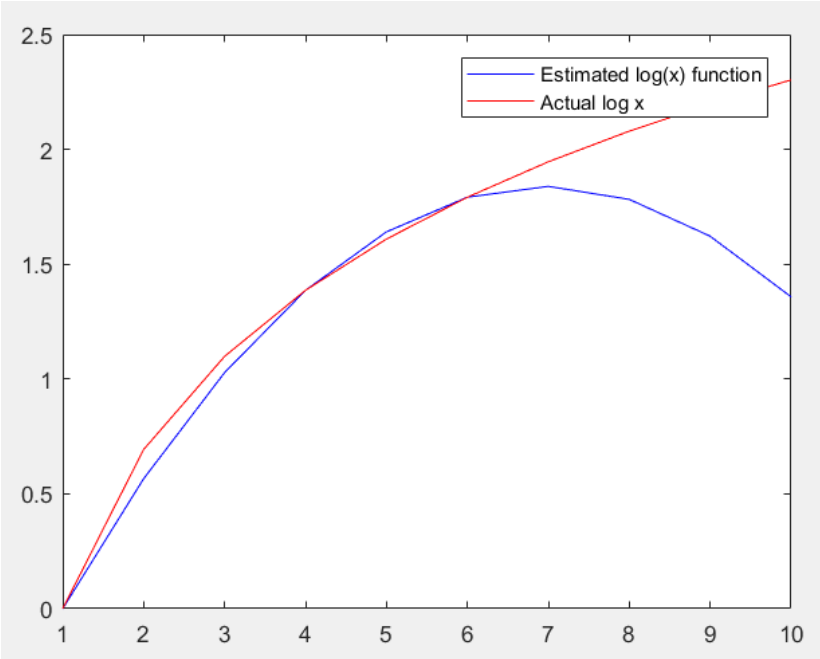
$$b_3 = \frac{\dfrac{1.791759 - 1.386294}{6 - 4} - 0.4620981}{6 - 1} = -0.0518731$$

Substituting these values into Eq. (17.6) yields the quadratic formula

$$f_2(x) = 0 + 0.4620981(x - 1) - 0.0518731(x - 1)(x - 4)$$

MATLAB Routine –

```
clc;clear all; close all;
x=1:1:10;
y = (-0.0518731*x.^2) + (0.7214635)*(x) - 0.6695904;
y1=log(x);
plot (x,y,'b');
hold on;
plot(x,y1,'r')
xlabel('x'); ylabel('log(x) function estimation using Newtons interpolation')
legend('actual log(x) function','estimated log(x) function')
```



LAGRANGE INTERPOLATING POLYNOMIAL

Suppose we formulate a linear interpolating polynomial as the weighted average of the two values that we are connecting by a straight line:

$$f(x) = L_1 f(x_1) + L_2 f(x_2) \tag{17.19}$$

where the L's are the weighting coefficients. It is logical that the first weighting coefficient is the straight line that is equal to 1 at x_1 and 0 at x_2:

$$L_1 = \frac{x - x_2}{x_1 - x_2}$$

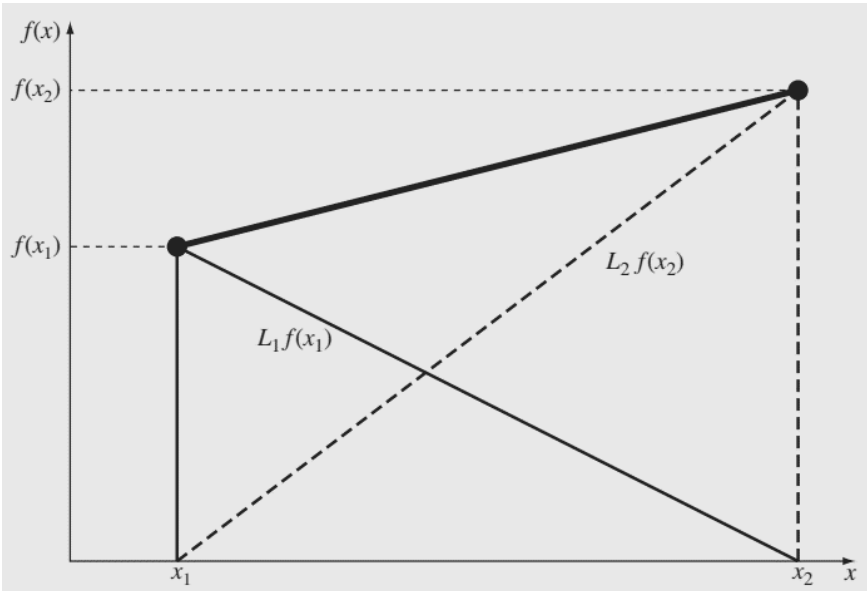
Similarly, the second coefficient is the straight line that is equal to 1 at x_2 and 0 at x_1:

$$L_2 = \frac{x - x_1}{x_2 - x_1}$$

Substituting these coefficients into Eq. (17.19) yields the straight line that connects the points (Fig. 17.8):

$$f_1(x) = \frac{x - x_2}{x_1 - x_2} f(x_1) + \frac{x - x_1}{x_2 - x_1} f(x_2) \tag{17.20}$$

where the nomenclature $f_1(x)$ designates that this is a first-order polynomial. Equation (17.20) is referred to as the *linear Lagrange interpolating polynomial.*



A visual depiction of the rationale behind Lagrange interpolating polynomials. The figure shows the first-order case. Each of the two terms of Eq. (17.20) passes through one of the points and is zero at the other. The summation of the two terms must, therefore, be the unique straight line that connects the two points.

The same strategy can be employed to fit a parabola through three points. For this case three parabolas would be used with each one passing through one of the points and equaling zero at the other two. Their sum would then represent the unique parabola that connects the three points. Such a second-order Lagrange interpolating polynomial can be written as

$$f_2(x) = \frac{(x - x_2)(x - x_3)}{(x_1 - x_2)(x_1 - x_3)} f(x_1) + \frac{(x - x_1)(x - x_3)}{(x_2 - x_1)(x_2 - x_3)} f(x_2)$$
$$+ \frac{(x - x_1)(x - x_2)}{(x_3 - x_1)(x_3 - x_2)} f(x_3) \tag{17.21}$$

Notice how the first term is equal to $f(x_1)$ at x_1 and is equal to zero at x_2 and x_3. The other terms work in a similar fashion.

where

$$L_i(x) = \prod_{\substack{j=1 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$$

where n = the number of data points and $\prod$ designates the "product of."

Problem Statement. Use a Lagrange interpolating polynomial of the first and second order to evaluate the density of unused motor oil at $T = 15$ °C based on the following data:

$x_1 = 0 \qquad f(x_1) = 3.85$

$x_2 = 20 \qquad f(x_2) = 0.800$

$x_3 = 40 \qquad f(x_3) = 0.212$

Solution. The first-order polynomial [Eq. (17.20)] can be used to obtain the estimate at $x = 15$:

$$f_1(x) = \frac{15 - 20}{0 - 20} 3.85 + \frac{15 - 0}{20 - 0} 0.800 = 1.5625$$

In a similar fashion, the second-order polynomial is developed as [Eq. (17.21)]

$$f_2(x) = \frac{(15 - 20)(15 - 40)}{(0 - 20)(0 - 40)} 3.85 + \frac{(15 - 0)(15 - 40)}{(20 - 0)(20 - 40)} 0.800$$
$$+ \frac{(15 - 0)(15 - 20)}{(40 - 0)(40 - 20)} 0.212 = 1.3316875$$

Assignment -

One numerical as suggested by Faculty.

Sample numerical.

17.4 Given the data

x	1	2	2.5	3	4	5
$f(x)$	0	5	7	6.5	2	0

(a) Calculate $f(3.4)$ using Newton's interpolating polynomials of order 1 through 3. Choose the sequence of the points for your estimates to attain the best possible accuracy. That is, the points should be centered around and as close as possible to the unknown.

(b) Repeat **(a)** but use the Lagrange polynomial.

Handwritten Notes

Given, $n+1$ data points $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$ with $x_0 < x_1 < \ldots < x_n$. "The lagrange polynomial is the nth degree polynomial passing through each point"

$$y = \frac{(x-x_1)(x-x_2)\ldots(x-x_n)}{(x_0-x_1)(x_0-x_2)\ldots(x_0-x_n)} y_0 + \frac{(x-x_0)(x-x_2)\ldots(x-x_n)}{(x_1-x_0)(x_1-x_2)\ldots(x_1-x_n)} y_1 + \ldots + \frac{(x-x_0)(x-x_1)\ldots(x-x_{n-1})}{(x_n-x_0)(x_n-x_1)\ldots(x_n-x_{n-1})} y_n$$

$$\therefore \quad y = \sum_{i=0}^{n} \left(y_i \prod_{\substack{j=0 \\ j \neq i}} \frac{x - x_j}{x_i - x_j} \right)$$

Proof of Lagrange theorem:

* n^{th} degree polynomial:

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots a_1 x^1 + a_0$$

* Let there be a nth degree polynomial

$$f(x) = A_0 (x_0 - x_1)(x_0 - x_2)(x - x_3) \ldots (x - x_n) + A_1 (x - x_0)(x - x_2)(x - x_3) \ldots (x - x_n) + \ldots + A_n (x - x_0)(x - x_1)(x - x_3) \ldots (x - x_{n-1})$$

substitute x_i to get A_i

put $x = x_0$ to get A_0

$$f(x_0) = y_0 = A_0 (x_0 - x_1)(x_0 - x_2)(x_0 - x_3) \ldots (x_0 - x_n)$$

$$A_0 = y_0 / (x_0 - x_1)(x_0 - x_2)(x_0 - x_3) \ldots (x_0 - x_n)$$

By substituting $x = x_1$ we get A_1

Numerical Solving:

Lagrange

1st order :

$$f(x) = f(x_0)\frac{(x-x_1)}{(x_0-x_1)} + f(x_1)\frac{(x-x_0)}{(x_1-x_0)}$$

2nd order :

$$f(x) = f(x_0)\frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)} + f(x_1)\frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)} + f(x_2)\frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}$$

3rd order

$$f(x) = \sum_{i=0}^{3} f(x_p) \prod_{\substack{j=0 \\ j\neq i}}^{3} \frac{x-x_j}{x_p-x_p}$$

Assume $f(1)=0$, $f(5)=0$ exists.

(Boundary Conditions)

Code and Output

```
clc; clear;

% given data
x = [1 2 2.5 3 4 5];
f = [0 5 7 6.5 2 0];

% linear (1st order)
for i = 2:5
    % using neighbors (i-1, i+1)
    L1 = (x(i) - x(i+1)) / (x(i-1) - x(i+1));
    L2 = (x(i) - x(i-1)) / (x(i+1) - x(i-1));

    Fr = L1 * f(i-1) + L2 * f(i+1);

```

```
    fprintf('Linear: Fx at %.2f = %.4f\n', x(i), Fr);
end


% second order
for i = 2:5

    % pick 3 points based on position
    if i < 4
        idx = [1 i+1 5];
    else
        idx = [1 i-1 5];
    end

    x0 = x(idx(1)); f0 = f(idx(1));
    x1 = x(idx(2)); f1 = f(idx(2));
    x2 = x(idx(3)); f2 = f(idx(3));

    % quadratic Lagrange basis
    L0 = ((x(i)-x1)*(x(i)-x2)) / ((x0-x1)*(x0-x2));
    L1 = ((x(i)-x0)*(x(i)-x2)) / ((x1-x0)*(x1-x2));
    L2 = ((x(i)-x0)*(x(i)-x1)) / ((x2-x0)*(x2-x1));

    F2 = f0*L0 + f1*L1 + f2*L2;

    fprintf('Second order: Fx at %.2f = %.4f\n', x(i), F2);
end


% third order
for i = 2:5

    % choose 4 points
    if i == 2
        idx = [1 3 4 5];
    elseif i == 3
        idx = [1 2 4 5];
    else
        idx = [i-1 i-2 i-3 5];
    end

    x0 = x(idx(1)); f0 = f(idx(1));
    x1 = x(idx(2)); f1 = f(idx(2));
    x2 = x(idx(3)); f2 = f(idx(3));
    x3 = x(idx(4)); f3 = f(idx(4));

    % cubic Lagrange basis
    L0 = (x(i)-x1)*(x(i)-x2)*(x(i)-x3) / ((x0-x1)*(x0-x2)*(x0-x3));
    L1 = (x(i)-x0)*(x(i)-x2)*(x(i)-x3) / ((x1-x0)*(x1-x2)*(x1-x3));
    L2 = (x(i)-x0)*(x(i)-x1)*(x(i)-x3) / ((x2-x0)*(x2-x1)*(x2-x3));
    L3 = (x(i)-x0)*(x(i)-x1)*(x(i)-x2) / ((x3-x0)*(x3-x1)*(x3-x2));

    F3 = f0*L0 + f1*L1 + f2*L2 + f3*L3;

    fprintf('Third order: Fx at %.2f = %.4f\n', x(i), F3);
end

```

Command Window

```
Linear: Fx at 2.00 = 4.6667
Linear: Fx at 2.50 = 5.7500
Linear: Fx at 3.00 = 5.3333
Linear: Fx at 4.00 = 3.2500
Second order: Fx at 2.00 = 6.0000
Second order: Fx at 2.50 = 6.8125
Second order: Fx at 3.00 = 6.6667
Second order: Fx at 4.00 = 2.0000
Third order: Fx at 2.00 = 6.1667
Third order: Fx at 2.50 = 6.3438
Third order: Fx at 3.00 = 7.6667
Third order: Fx at 4.00 = 2.0000
```